

SEND5232: Software Architecture and Design Project

Bereket Kiros – ugr/189518/16 – Section 2

PHASE 1 — ARCHITECTURAL STRUCTURE & INITIAL DESIGN

Lihq'et Bookstore System

1. System Purpose

The Lihq'et Bookstore System is a full-stack web-based application designed to manage bookstore inventory efficiently through a digital interface. The system allows authorized users (bookstore administrators and staff) to perform CRUD (Create, Read, Update, Delete) operations on book records stored in a relational database.

The system improves traditional manual inventory management by providing a structured, real-time, and interactive platform for managing book data.

2. System Scope

The scope of the system includes:

Included:

- Book management (add, update, delete, view books)
- RESTful API communication between frontend and backend
- Persistent data storage using MySQL
- Interactive web-based user interface
- Basic inventory tracking (title, author, price, stock)

Not Included:

- Payment processing
 - Online ordering system
 - User authentication (future extension)
 - Customer accounts (future extension)
-

3. Architectural Style

The system is designed using:

◆ 3-Tier Layered Architecture + MVC + Service Layer Pattern

Layers:

1. Presentation Layer (Frontend)

- HTML, CSS, JavaScript
- Handles UI rendering and user interaction
- Communicates with backend via REST API (Fetch)

2. Application Layer (Backend - Node.js + Express)

This layer follows **MVC + Service Layer architecture**:

MVC Breakdown:

- **Model Layer** - MySQL database tables - Data schema (Books table)
- **View Layer** - Not server-rendered (frontend handles view) - API returns JSON only
- **Controller Layer** - Express routes (/books) - Handles HTTP requests/responses

Service Layer

- Contains business logic and Handles:
 - validation rules
 - data processing
 - DB interaction coordination

This ensures controllers stay clean (**Separation of Concerns**)

3. Data Layer (DB)

- MySQL database
 - Stores persistent book records
 - Executes SQL queries
-

4. Key Components / Modules

4.1 Frontend Module

Responsible for user interaction.

Components: - Book Table UI - Add Book Form - Edit/Delete Buttons - Fetch API handler (app.js)

Responsibilities: Display book data, Send user requests to backend, Update UI dynamically

4.2 Backend Module (MVC + Service Layer)

Controller Layer

- bookController.js
- Handles routes: GET /books, POST /books, PUT /books/:id and DELETE /books/:id

Service Layer

- bookService.js
- Contains: business logic, validation rules and database operations coordination

Model Layer

- bookModel.js (or direct MySQL queries)
- Responsible for DB interaction

4.3 Database Module

- MySQL database
- Table: books

Attributes:

- id (PK)
 - title
 - author
 - price
 - stock
-

5. System Interaction

The system follows this flow:

1. User interacts with frontend (e.g., clicks “Add Book”)
2. Frontend sends HTTP request to backend API
3. Controller receives request
4. Controller calls Service Layer
5. Service processes logic + validates data
6. Service interacts with Model (MySQL)
7. Database executes SQL query
8. Response returned back through layers
9. Frontend updates UI dynamically

6. Architectural Views and design principles

Design Principles Applied

- **Separation of Concerns (SoC)** → layers separated
- **DRY** → reusable service logic
- **KISS** → simple CRUD design
- **Low Coupling** → frontend independent of DB
- **High Cohesion** → each module has one responsibility
- **Information Hiding** → DB hidden behind service layer

A. Context Diagram (SYSTEM LEVEL VIEW)

System: Lihq'et Bookstore System

External Actors:

- Admin / Staff User and MySQL Database

Interactions:

- User → System: CRUD requests
- System → Database: SQL queries
- Database → System: query results
- System → User: JSON responses/UI updates

Lihq'et Bookstore System - Context Diagram

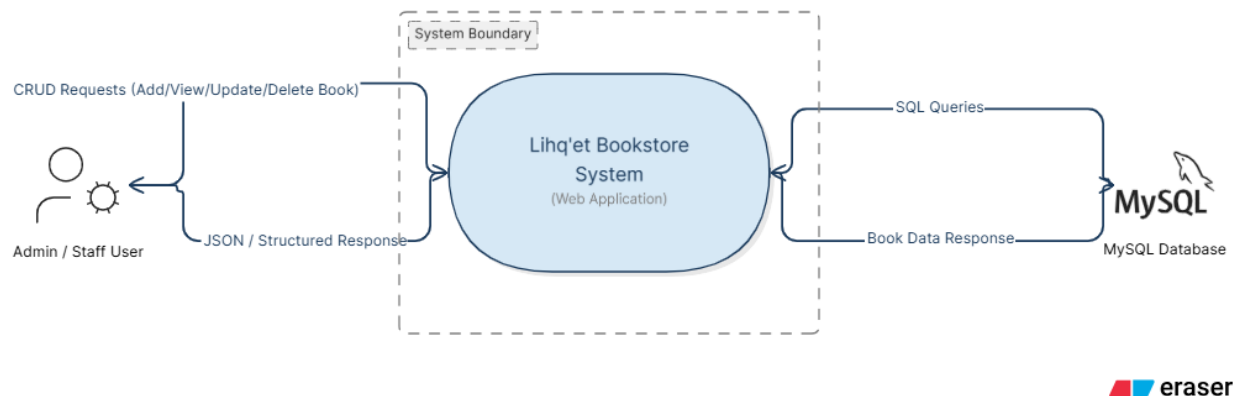


Figure 1: Context Diagram of Lihq' et Bookstore System

The context diagram illustrates the Lihq'et Bookstore System as a single high-level entity and shows its interaction with external actors. The primary external actor is the Admin/Staff User, who interacts with the system by sending CRUD (Create, Read, Update, Delete) requests such as adding, viewing, updating, and deleting book records. The system processes these requests and communicates with the MySQL database to store and retrieve data. The database returns the requested information, and the system responds to the user with structured data (JSON) displayed through the user interface. This diagram emphasizes the system boundary and highlights the flow of data between the system and its external environment without exposing internal implementation details.

B. Component Diagram (LOGICAL VIEW)

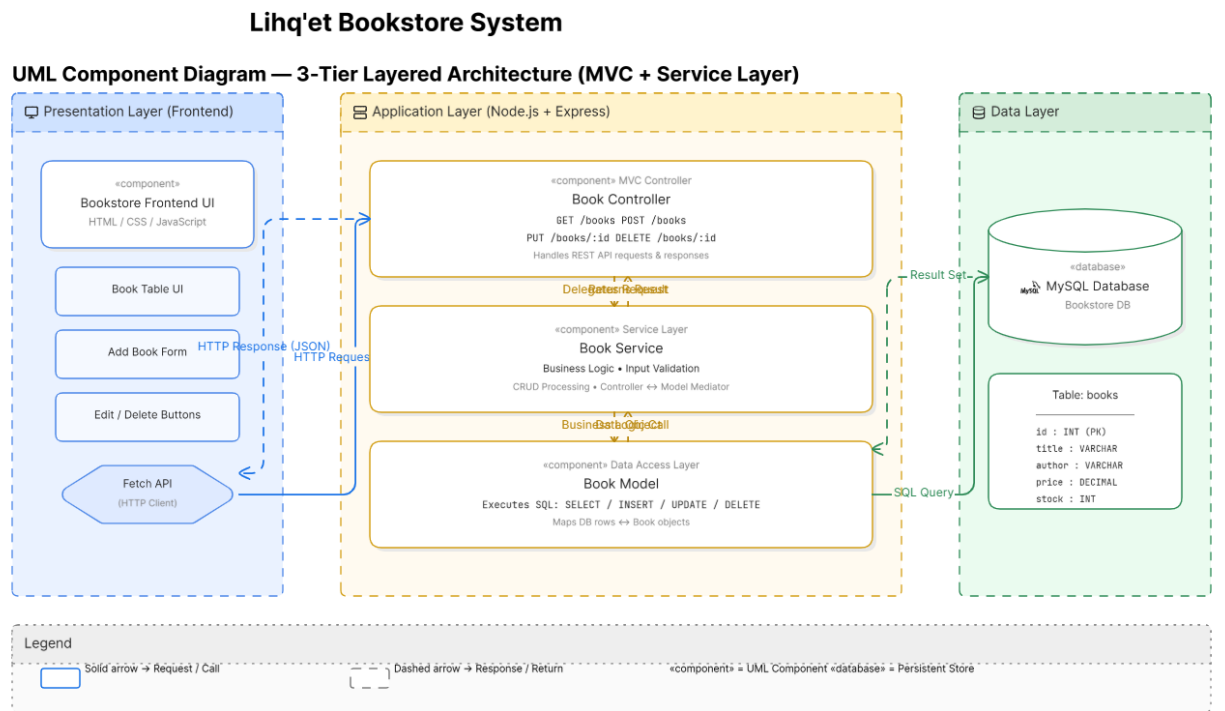


Figure 2: Component Diagram of Lihq' et Bookstore System

The component diagram presents the internal logical structure of the Lihq'et Bookstore System based on a 3-tier layered architecture combined with the MVC and Service Layer patterns. The frontend component, built using HTML, CSS, and JavaScript, is responsible for user interaction and communicates with the backend through HTTP requests. The backend is divided into three main components: the Controller, Service Layer, and Model. The Controller handles incoming API requests and forwards them to the Service Layer, which contains the business logic and validation rules. The Service Layer interacts with the Model component, which is responsible for executing SQL queries and communicating with the MySQL database. The database stores and retrieves book records. This layered separation ensures low coupling, high cohesion, and maintainability while clearly illustrating how data flows between components.

C. Deployment Diagram (PHYSICAL VIEW)

Nodes:

Client Machine

- Browser (Frontend UI)

Server Machine

- Node.js + Express App
- Service Layer
- Controller Layer

Database Server

- MySQL Database

Communication:

- HTTP/HTTPS between Client ↔ Server
 - SQL connection between Server ↔ Database
-

Lihq'et Bookstore System - UML Deployment Diagram

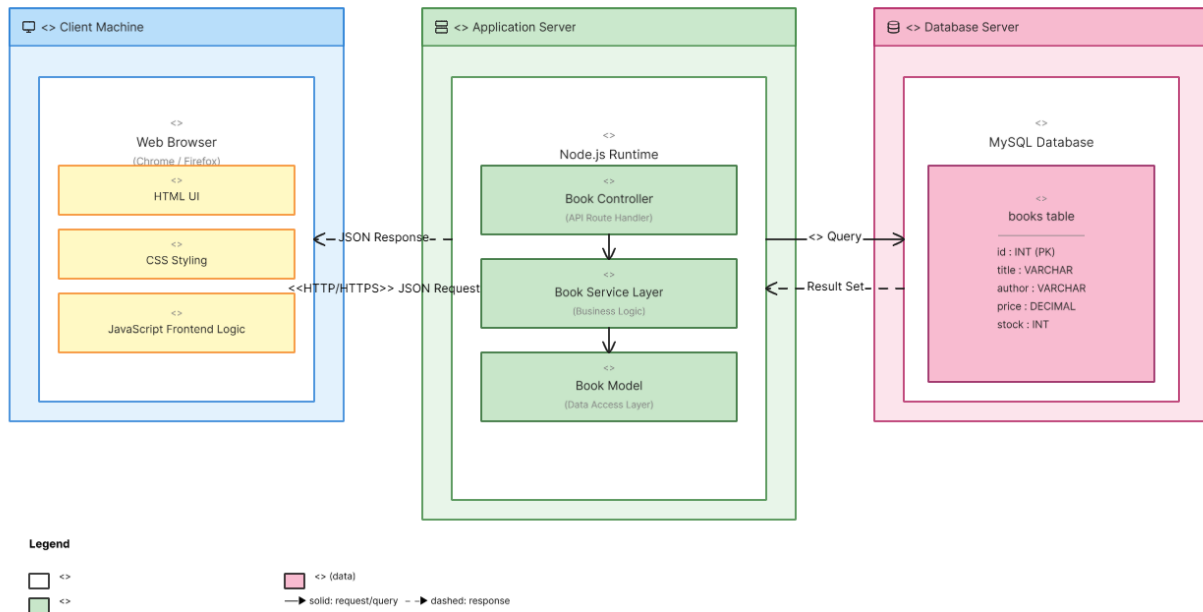


Figure 3: Deployment Diagram of Lihq' et Bookstore System

The deployment diagram shows the physical structure of the Lihq'et Bookstore System across three main nodes: the Client Machine, Application Server, and Database Server. The client runs the frontend in a web browser, while the server hosts the backend using Node.js and Express, including the Controller, Service, and Model components. The database server runs MySQL to store book data. Communication occurs via HTTP/HTTPS between client and server, and SQL between the server and database, illustrating clear separation and interaction between system components.